Verifying Distributed Algorithms with Interactive Theorem Provers: A Survey of Approaches

MAXWELL MOIR, University of Canterbury, New Zealand

Formal verification is a topical field in modern Computer Science. Proving the correctness of programs is not only elegant in a theoretical sense, but is often necessary in mission-critical applications. Due to the difficulty in formalising programs, reliability in industry is often left to automated testing. However, in specific circumstances where correctness must be ensured, formal proofs may be required.

CCS Concepts: • **Theory of computation → Logic and verification**.

Additional Key Words and Phrases: distributed systems, formal verification, proof assistants

ACM Reference Format:
Maxwell Moir. 2026. Verifying Distributed Algorithms with Interactive Theorem Provers: A Survey of Approaches. *J. ACM* 000, 0, Article 000 (2026), 6 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

In an increasingly interconnected world, distributed algorithms play an integral role in much of the software we use daily. Ensuring these algorithms function as intended protects against failures that may be technically, economically, or even medically critical.

1.1 Motivation

Distributed systems are famously difficult to implement correctly. Their divided and concurrent nature can often lead to subtle bugs that are difficult to reproduce. Additionally, errors can propagate throughout a system, and partial failures can lead to unexpected negative outcomes. For this reason, it is often unrealistic to achieve total coverage with traditional testing. Formal verification can provide a strong guarantee of correctness in these situations. Interactive theorem-proving as a verification method is specifically strong for distributed systems, as it can verify correctness in all states without iteratively exploring them. theorem-proving also allows for flexibility in the parameterisation of systems, such as verifying an algorithm for any number of nodes, N.

1.2 Contributions

For this reason, this survey will explore the verification of distributed algorithms with interactive theorem provers. It provides a taxonomy of different theorem-based verification approaches and frameworks, reviews several common tools, and contrasts with other verification approaches. In addition, this survey will contrast the theorem-proving verification approach against other common methods, such as manual proofs and model checking.

Author's address: Maxwell Moir, University of Canterbury, Christchurch, New Zealand, mmo199@uclive.ac.nz.

ACM 0004-5411/2026/0-ART000
https://doi.org/XXXXXXX.XXXXXXX

2 METHODOLOGY

2.1 Literature Search Strategy

To identify relevant literature, this survey employed a systematic literature search strategy. Two major academic databases were selected: the ACM Digital Library and IEEE Xplore. These databases were chosen for their broad coverage of research in formal verification and distributed systems. They were accessed on April 17 2026.

To ensure the quality of the review, the PRISMA framework was used to inform the search strategy. The strategy consisted of constructing a query followed by database searching and screening. First, relevant keywords were curated based on the topic of this survey, and a search query was constructed. The query used is as follows.

> ("proof assistant" OR "theorem prover" OR "mechanized verification" OR "formal verification") AND ("distributed protocol" OR "distributed algorithm")

Selected papers were limited to peer-reviewed English-language publications. There was no strict lower bound imposed on the publication date, although results were naturally concentrated in the last 20 years. This query yielded 213 results from the ACM Digital Library and 49 results from IEEE Xplore, which were exported for further screening.

2.2 Inclusion and Exclusion Criteria

These results were then screened by title and abstract with standard inclusion and exclusion requirements. Included papers focused on the verification of distributed algorithms using theorem-proving. Duplicate papers within and between databases were removed. Additionally, papers were excluded if model checking or manual proof were the primary verification technique. This reduced the number of results to 37 for ACM and 15 for IEEE Xplore.

2.3 Citation Chaining

Finally, the reference lists of the selected papers were scanned,and fundamental or commonly cited papers were added. Google Scholar was used as a supplementary tool for citation chaining. Any additional papers identified this way were included only if they met the above criteria or were primarily concerned with a related topic to be explained.

3 PRELIMINARY

3.1 Distributed Algorithms

Distributed computing is a fundamental field in Computer Science. It concerns intercommunication among distinct autonomous components in a system. These patterns often arise, usually when there is a network of multiple devices.

There are several common challenges in distributed computing that are often the targets of formal verification. For example, consensus protocols such as Paxos Another example is Byzantine Fault Tolerance.

3.2 Interactive Theorem Provers

Interactive Theorem Provers, or proof assistants, have seen a recent surge in popularity among computer scientists and mathematicians alike. Proof assistants are tools that assist the human development of formal proofs. These formal proofs can then be automatically checked by the machine, ensuring correctness at each stage of the process. Theorem provers are extremely useful in the formal verification of protocols, where they can be used to ensure specific properties of a system hold. For this reason, proof assistants such as Coq, Lean, Isabelle, and Dafny have become popular for verifying distributed algorithms.

3.3 Verification

Verifying the correctness of a program requires that specific conditions about a system can be shown to hold in certain states. These conditions can be arbitrarily chosen, but usually represent ... The two most notable property categories of a system are “Safety” and “Liveness”. A safety property ensures protection against unfavourable events, whereas liveness properties make sure that the program functions as intended.

Fault tolerance is another notable aspect of ensuring a program works as intended. Byzantine fault tolerance, specifically, is a common problem in

4 APPROACHES TO VERIFYING DISTRIBUTED ALGORITHMS

4.1 Proof techniques

The primary proof technique used in the mechanised theorem-proving approach for formal verification is inductive invariants. When using this approach

Inductive invariants

Temporal logic-based

Refinement based Compositional [16] Automation

4.2 Tools and Common frameworks

There are many languages and frameworks used to verify distributed systems. The most commonly used tool in the literature is Rocq, formerly Coq. There are multiple frameworks implemented for Rocq that specifically focus on distributed systems, such as Verdi [15], and Diesel [12]. These ...

Another regularly used theorem prover is TLAPS [2]

Iron Fleet [6]

A more contemporary tool is Lean4. There is less research in verification with Lean4, as it is a younger tool. In 2025, the Veil [9] framework was introduced as a basis to verify distributed protocols specifically.

These languages and frameworks all have advantages and disadvantages.

5 CASE STUDIES

5.1 Consensus Algorithms

Consensus algorithms are used to agree on one result in a group of multiple participants. They typically assume that participants can crash, but will not act maliciously. Two well-known solutions to the consensus problem are the Paxos and Raft protocols. The Paxos protocol, originally specified by Lamport [4], is one method to facilitate consensus.

Consensus is also an important problem in application. For example, in blockchain systems, consensus is required between distributed, and potentially faulty, machines to ensure users can agree on the state of the system. In their 2018 paper [10], Pirlea and Sergey present a blockchain-based distributed consensus protocol, verified in the Rocq proof assistant. This is an example of a modern application of a formally verified distributed protocol, in which correctness is absolutely critical for user trust in the system.

AWS [1] Mongo [11]

5.2 Byzantine Fault Tolerance

Byzantine fault tolerance (BFT) protocols aim to achieve consensus in a group of nodes containing potentially malicious members. Consensus protocols, such as Paxos and Raft, assume a crash-resistant model, but cannot handle unspecified node behaviour. BFT protocols aim to protect against this. This problem was also introduced by Lamport et al. [7].

BFT [16]

Once again, this problem arises in blockchain systems. Blockchain Byzantine [13]

5.3 Mutual Exclusion

The mutual exclusion problem is another classic challenge in distributed computing. It was originally posed by Dijkstra in a 1965 article [5] For a protocol to satisfy mutual exclusion, no two independent processes may be executing their so called "critical portion" at the same time.

An example of the mutual exclusion property formally verified can be found in the Verdi specification [15]. Here, a toy lock service application is implemented in Verdi, with mutual exclusion verified as one of the correctness properties. The mutual exclusion property is expressed as a predicate and is shown to hold under all traces produced under a given semantics.

6 COMPARISON OF VERIFICATION STRATEGIES

There are three major verification paradigms that are often used for distributed algorithm verification. The other two predominant methods of formally verifying distributed systems are model checking and manual proofs. Each of these other verification approaches has its own advantages and disadvantages in relation to theorem-proving.

Many solutions to the previously mentioned problems are based on manual proofs. For example, De Prisco et al. [3] developed an I/O automaton model and verified the Paxos algorithm. The paper contains a manual correctness proof, in which the validity of each component is proved with invariants. Manual proofs are more expressive than mechanised proofs, but have a weaker correctness guarantee because they depend on mathematical arguments formulated by humans. This means they are more prone to errors in their development or implementation, and are more difficult to maintain when scaling.

Model checking is another approach for verifying the correctness of a distributed system. Model checking expands the state space, exhaustively exploring every possible state and ensuring that the correctness properties hold in each one. This was exemplified by Tsuchiya and Springer [14], who used bounded model checking to verify two consensus algorithms, including Hybrid-1, an improved version of the Fast Paxos algorithm. They were able to mechanically verify agreement up to 8 processes and termination up to 11 processes. A major limitation of model checking is the state space explosion problem, which restricts it to bounded or specifically abstracted models. This often reduces the maximum number of processes in a verified protocol.

A theorem-proving approach does not have this issue, as it can ensure that correctness conditions hold for any finite number of processes.

7 CHALLENGES AND FUTURE DIRECTIONS

One of the common criticisms of theorem-based formal verification is that it is too time-consuming to apply practically. In addition to this, verification requires specialised engineers, often with a mathematical background.

Another common challenge is scalability. After a system has been formally verified, ongoing maintainance is required to reverify it each time it changes.

There are differing perspectives on this .

Increased automation

One potentially direction of future research is the application of machine learning to assist with formalisation. In a 2023 paper [8], Liu et al. propose a technique, based on reinforcement learning for predicting contract invariants that are useful for proving arithmetic safety. Although this paper doesn't focus on a distributed protocol, it highlights the potential that ML may hold in reducing the verification time of complex systems.

8 CONCLUSION

This survey has provided an overview of the use of interactive theorem provers for the formal verification of distributed systems. It has covered common languages, frameworks, proof techniques and algorithms in the area and discusses their relative advantages and disadvantages. Throughout the paper, several patterns have repeatedly manifested. For example, the trade-offs between development time and degree of correctness guaranteed. Case studies in consensus agreement and Byzantine fault tolerance have demonstrated the feasibility and importance of protocol verification.

Due to common intercacies in distributed algorithms, correctness guarantees can be especially valuable. In relation to manual proofs and model checking, interactive theorem provers act as a powerful tool in reasoning about the correctness of these systems.

REFERENCES

[1] Marc Brooker and Ankush Desai. 2025. Systems Correctness Practices at Amazon Web Services. *Commun. ACM* 68, 6 (June 2025), 38–42. https://doi.org/10.1145/3729175

[2] Denis Cousineau, Damien Doligez, Leslie Lamport, Stephan Merz, Daniel Ricketts, and Hernan Vanzetto. 2012. TLA+ Proofs. *Proceedings of the 18th International Symposium on Formal Methods (FM 2012), Dimitra Giannakopoulou and Dominique Mery, editors. Springer-Verlag Lecture Notes in Computer Science* 7436 (Jan. 2012), 147–154.

[3] Roberto De Prisco, Butler Lampson, and Nancy Lynch. 1997. Revisiting the Paxos Algorithm. In *Distributed Algorithms*, Gerhard Goos, Juris Hartmanis, Jan Van Leeuwen, Marios Mavronicolas, and Philippas Tsigas (Eds.). Vol. 1320. Springer Berlin Heidelberg, Berlin, Heidelberg, 111–125. https://doi.org/10.1007/BFb0030679

[4] Digital Equipment Corporation and Leslie Lamport. 2019. *The Part-Time Parliament.* Association for Computing Machinery. https://doi.org/10.1145/3335772.3335939

[5] E. W. Dijkstra. 1965. Solution of a Problem in Concurrent Programming Control. *Commun. ACM* 8, 9 (Sept. 1965), 569. https://doi.org/10.1145/365559.365617

[6] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath Setty, and Brian Zill. 2015. IronFleet: Proving Practical Distributed Systems Correct. In *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP '15)*. Association for Computing Machinery, New York, NY, USA, 1–17. https://doi.org/10.1145/2815400.2815428

[7] Leslie Lamport, Robert Shostak, and Marshall Pease. 2019. The Byzantine Generals Problem. In *Concurrency: The Works of Leslie Lamport*. Association for Computing Machinery, New York, NY, USA, 203–226.

[8] Junrui Liu, Yanju Chen, Bryan Tan, Isil Dillig, and Yu Feng. 2023. Learning Contract Invariants Using Reinforcement Learning. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22)*. Association for Computing Machinery, New York, NY, USA, 1–11. https://doi.org/10.1145/3551349.3556962

[9] George Pîrlea, Vladimir Gladshtein, Elad Kinsbruner, Qiyuan Zhao, and Ilya Sergey. 2025. Veil: A Framework for Automated and Interactive Verification of Transition Systems. In *Computer Aided Verification: 37th International Conference, CAV 2025, Zagreb, Croatia, July 23-25, 2025, Proceedings, Part III*. Springer-Verlag, Berlin, Heidelberg, 26–41. https://doi.org/10.1007/978-3-031-98682-6_2

[10] George Pîrlea and Ilya Sergey. 2018. Mechanising Blockchain Consensus. In *Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs*. ACM, Los Angeles CA USA, 78–90. https://doi.org/10.1145/3167086

[11] William Schultz, Ian Dardik, and Stavros Tripakis. 2022. Formal Verification of a Distributed Dynamic Reconfiguration Protocol. In *Proceedings of the 11th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2022)*. Association for Computing Machinery, New York, NY, USA, 143–152. https://doi.org/10.1145/3497775.3503688

[12] Ilya Sergey, James R. Wilcox, and Zachary Tatlock. 2017. Programming and Proving with Distributed Protocols. *Proc. ACM Program. Lang.* 2, POPL (Dec. 2017), 28:1–28:30. https://doi.org/10.1145/3158116

[13] Pierre Tholoniat and Vincent Gramoli. 2022. Formal Verification of Blockchain Byzantine Fault Tolerance. In *Handbook on Blockchain*, Duc A. Tran, My T. Thai, and Bhaskar Krishnamachari (Eds.). Vol. 194. Springer International Publishing, Cham, 389–412. https://doi.org/10.1007/978-3-031-07535-3_12

[14] Tatsuhiro Tsuchiya and André Schiper. 2008. Using Bounded Model Checking to Verify Consensus Algorithms. In *Proceedings of the 22nd International Symposium on Distributed Computing (DISC '08)*. Springer-Verlag, Berlin, Heidelberg, 466–480. https://doi.org/10.1007/978-3-540-87779-0_32

[15] James R. Wilcox, Doug Woos, Pavel Panchekha, Zachary Tatlock, Xi Wang, Michael D. Ernst, and Thomas Anderson. 2015. Verdi: A Framework for Implementing and Formally Verifying Distributed Systems. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '15)*. Association for Computing Machinery, New York, NY, USA, 357–368. https://doi.org/10.1145/2737924.2737958

[16] Qiyuan Zhao, George Pîrlea, Karolina Grzeszkiewicz, Seth Gilbert, and Ilya Sergey. 2024. Compositional Verification of Composite Byzantine Protocols. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. Association for Computing Machinery, New York, NY, USA, 34–48. https://doi.org/10.1145/3658644.3690355